

Name: Vikash Kumar || Business Case: Target SQL

Problem Statement:

1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

1. Data type of all columns in the "customers" table.

Screenshot of output

The screenshot shows the Google BigQuery Explorer interface. On the left, the 'Explorers' pane shows a project named 'scaler-dsml-sql-456021' with a dataset 'target' containing tables: customers, geolocation, order_items, order_reviews, orders, payments, products, and sellers. The 'customers' table is selected. The main pane shows the 'Schema' tab for the 'customers' table. It displays a table with columns: Field name, Type, Mode, Key, Collation, Default Value, Policy Tags, and Description. The data is as follows:

Field name	Type	Mode	Key	Collation	Default Value	Policy Tags	Description
customer_id	STRING	NULLABLE	-	-	-	-	-
customer_unique_id	STRING	NULLABLE	-	-	-	-	-
customer_zip_code_prefix	INTEGER	NULLABLE	-	-	-	-	-
customer_city	STRING	NULLABLE	-	-	-	-	-
customer_state	STRING	NULLABLE	-	-	-	-	-

2. Get the time range between which the orders were placed.

Query:

```
select min(order_purchase_timestamp) as first_order,  
       max(order_purchase_timestamp) as latest_order  
from `target.orders`
```

Screenshot of output

The screenshot shows the Google BigQuery Explorer interface with a query executed. The query is: `select min(order_purchase_timestamp) as first_order, max(order_purchase_timestamp) as latest_order from `target.orders``. The results are displayed in a table with two columns: first_order and latest_order. The first row shows the results: 2016-09-04 21:15:19 UTC and 2018-10-17 17:30:18 UTC.

Job information	Results	Chart	JSON	Execution details	Execution graph
Row	first_order	latest_order			
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC			

Insights : By finding the first order and latest order we get to know how long the business has been active in processing orders.

3. Count the Cities & States of customers who ordered during the given period

Query :

```
select count(distinct c.customer_city) as city_count,  
       count(distinct c.customer_state) as state_count  
from `target.orders` o  
inner join `target.customers` c  
on o.customer_id=c.customer_id
```

Screenshot of output

The screenshot displays the Google BigQuery web interface. On the left, the 'Explorer' panel shows a project named 'scaler-dsml-sql-456021' with a dataset 'target' containing tables like 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'sellers'. The main editor shows a SQL query to count distinct cities and states from the 'target.orders' table, joined with 'target.customers'. Below the query editor, a message states 'This query will process 8 MB when run.' The 'Query results' section shows a table with two columns: 'city_count' and 'state_count'. The first row shows a count of 4119 for cities and 27 for states. The bottom of the interface shows 'Results per page: 50' and '1 - 1 of 1'.

Row	city_count	state_count
1	4119	27

Insights :

- 1) By sorting the query result by count of city and states, it is observed that City= "sao paulo", state="SP" has max no. of customers who have made the orders .
- 2) Out of Total 8011 cities, only 4119 are seeing order from customers.

Recommendations:

- 1) We should encourage customers to make purchase through Target by doing some marketing strategies over the city from which orders are not coming.

2. In-depth Exploration:

1. Is there a growing trend in the no. of orders placed over the past years?

Query :

```
select
    extract(year from order_purchase_timestamp) as order_year,
    count(order_id) as no_of_orders
from `target.orders`
group by extract(year from order_purchase_timestamp)
order by order_year
```

Screenshot of output

The screenshot displays a BigQuery workspace. On the left, the 'Explorer' panel shows a search for 'target' with two results: 'customers' and 'geolocation'. The main editor shows a query titled 'Untitled query' with the following SQL code:

```
-- 1. Is there a growing trend in the no. of orders placed over the past years?
select
    extract(year from order_purchase_timestamp) as order_year,
    count(order_id) as no_of_orders
from `target.orders`
group by extract(year from order_purchase_timestamp)
order by order_year
```

Below the query editor, a message indicates 'Query completed'. The 'Query results' section shows a table with the following data:

Row	order_year	no_of_orders
1	2016	329
2	2017	45101
3	2018	54011

The interface also includes a 'Repository' panel on the left, a 'Job history' section at the bottom, and a 'Results per page' dropdown set to 50.

Insights :

- 1) By sorting the query over the order by years it is clearly seen that there is a growing trend in the no. of orders placed over the past years
- 2) From 2016 to 2017 there was a major spike in the no. of orders as seen in the result.

Recommendations:

- 1) We should try Investing more in scaling operations, improving infrastructure and exploring new markets.

2. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Query:

```
with monthly_orders as
(
  select
    extract(month from order_purchase_timestamp) as order_month,
    count(order_id) as no_of_orders
  from `target.orders`
  group by extract(month from order_purchase_timestamp)
  order by order_month),

monthly_seasonality as
(
  select order_month, no_of_orders,
    case
      when order_month in (1,2,12) then "Winter"
      when order_month in (3,4,5) then "Summer"
      when order_month in (6,7,8) then "Monsoon"
      when order_month in (9,10,11) then "Autumn"

    END as season
  from monthly_orders
)

select season, sum(no_of_orders) as season_wise_orders
from monthly_seasonality
group by season
order by season_wise_orders desc
```

Screenshot of output

The screenshot displays the Google BigQuery web interface. On the left, the 'Explorer' panel shows a search for 'target' with two results: 'customers' and 'geolocation'. The main editor shows a SQL query named 'Untitled query' that defines a CTE 'monthly_orders' and a CTE 'monthly_seasonality' which categorizes months into seasons. Below the query, a message states 'This query will process 3.98 MB when run.' The 'Query results' section is active, showing a table with 4 rows and 2 columns: 'season' and 'season_wise_orders'. The results are ordered by the number of orders in descending order.

Row	season	season_wise_orders
1	Monsoon	30573
2	Summer	29809
3	Winter	22251
4	Autumn	16808

Insights :

- 1) By figuring out the orders count by monthly seasonality and sorting the query over the season wise orders it is clearly seen that Monsoon season sees highest no. of orders followed by summer, winter and Autumn which sees lowest no. of orders.

Recommendations:

- 1) We should try to align marketing, inventory, staffing and logistics to match peak demand cycles which is in Monsoon season.
- 2) Also, we can Run **seasonal promotions** during high-performing months.
3. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)
 1. 0-6 hrs : Dawn
 2. 7-12 hrs : Mornings
 3. 13-18 hrs : Afternoon
 4. 19-23 hrs : Night

Query:

```
with time_orders as
(
    select
        extract(hour from order_purchase_timestamp) as order_time,
        count(order_id) as no_of_orders
    from `target.orders`
    group by extract(hour from order_purchase_timestamp)
    order by order_time),

hour_orders as
(
    select order_time, no_of_orders,
        case
            when order_time between 0 and 6 then "Dawn"
            when order_time between 7 and 12 then "Mornings"
            when order_time between 13 and 18 then "Afternoon"
            when order_time between 19 and 23 then "Night"
        END as time_of_day
    from time_orders)

select time_of_day, sum(no_of_orders) as total_orders_by_time_of_day
from hour_orders
group by time_of_day
order by total_orders_by_time_of_day desc
```

Screenshot of output

The screenshot displays the Google BigQuery web interface. On the left, the 'Explorer' panel shows a search for 'target' with two results: 'customers' and 'geolocation'. The main editor shows a SQL query that categorizes orders by time of day and calculates the total number of orders for each category. The query results are displayed in a table with columns 'time_of_day' and 'total_orders_by_time'. The results show 38135 orders in the Afternoon, 28331 in the Night, 27733 in the Mornings, and 5242 in the Dawn.

```
12   when order_time between 0 and 6 then "Dawn"
13   when order_time between 7 and 12 then "Mornings"
14   when order_time between 13 and 18 then "Afternoon"
15   when order_time between 19 and 23 then "Night"
16   END as time_of_day
17   from time_orders)
18
19 select time_of_day, sum(no_of_orders) as total_orders_by_time_of_day
20 from hour_orders
21 group by time_of_day
22 order by total_orders_by_time_of_day desc
```

Query results

Row	time_of_day	total_orders_by_time
1	Afternoon	38135
2	Night	28331
3	Mornings	27733
4	Dawn	5242

Insights :

- 1) By figuring out the total orders count by time of the day and sorting the query over the time_of_day it is clearly seen that Most of the orders are being Placed in Afternoon.
- 2) Also at Dawn less no. of orders are being placed as seen from output of the query.

Recommendations:

- 1) We should try to Schedule email campaigns and run ads in peak hours for higher visibility in Morning and Dawn
- 2) Also, we can Run ads to target users with night owl promos.
- 3) We can provide some offers/coupons to increase no. of orders at Dawn.

3. Evolution of E-commerce orders in the Brazil region:

1. Get the month on month no. of orders placed in each state.

Query:

```
select c.customer_state,
       extract(month from o.order_purchase_timestamp) as order_month,
       count(o.order_id) as no_of_orders
from `target.orders` o
inner join `target.customers` c
on o.customer_id=c.customer_id
group by c.customer_state,order_month
order by c.customer_state,order_month
```

Screenshot of output

The screenshot shows the Google BigQuery interface. On the left is the Explorer panel with a search for 'target' and a list of datasets including 'target', 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'sellers'. The main panel displays an 'Untitled query' with the following SQL code:

```
-- Get the month on month no. of orders placed in each state.
select c.customer_state,
       extract(month from o.order_purchase_timestamp) as order_month,
       count(o.order_id) as no_of_orders
```

Below the query editor, a message indicates 'Query completed'. The 'Query results' section shows a table with the following data:

Row	customer_state	order_month	no_of_orders
1	SP	8	4982
2	SP	5	4632
3	SP	7	4381
4	SP	6	4104
5	SP	3	4047
6	SP	4	3967
7	SP	2	3357
8	SP	1	3351
9	SP	11	3012
10	SP	12	2357

At the bottom right, it indicates 'Results per page: 50' and '1 - 50 of 322'.

Insights :

- 1) By figuring out the month on month state wise no. of orders it seems customer state with name "SP" as max. no of orders as compared to other states in each of its month
- 2) Month on month data helps to identify regional seasonality or any declining interest.
- 3) It helps to figure out the States with declining or stagnant growth

Recommendations:

- 1) Company can Investigate causes like delivery delays, fewer promotions, low engagement for the month with lower no. of orders.

2) How are the customers distributed across all the states?

For distribution of customers across all states we check (i) no. of customers in each state and (ii) Only Customers Who Have Placed at Least One Order per State

Query:

```
select customer_state,
       count(customer_id) as total_customers
from `target.customers`
group by customer_state
order by total_customers desc
```

Screenshot of output

The screenshot shows the Google Cloud BigQuery interface. On the left is the Explorer pane with a search for 'target' and a list of datasets including 'target', 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'sellers'. The main pane displays an 'Untitled query' with the following SQL code:

```
select customer_state,
       count(customer_id) as total_customers
from `target.customers`
group by customer_state
order by total_customers desc
```

Below the query, a message states: 'This query will process 3.6 MB when run.' The 'Query results' section shows a table with two columns: 'customer_state' and 'total_customers'. The results are sorted in descending order of total customers.

Row	customer_state	total_customers
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637
7	BA	3380
8	DF	2140
9	ES	2033
10	GO	2020

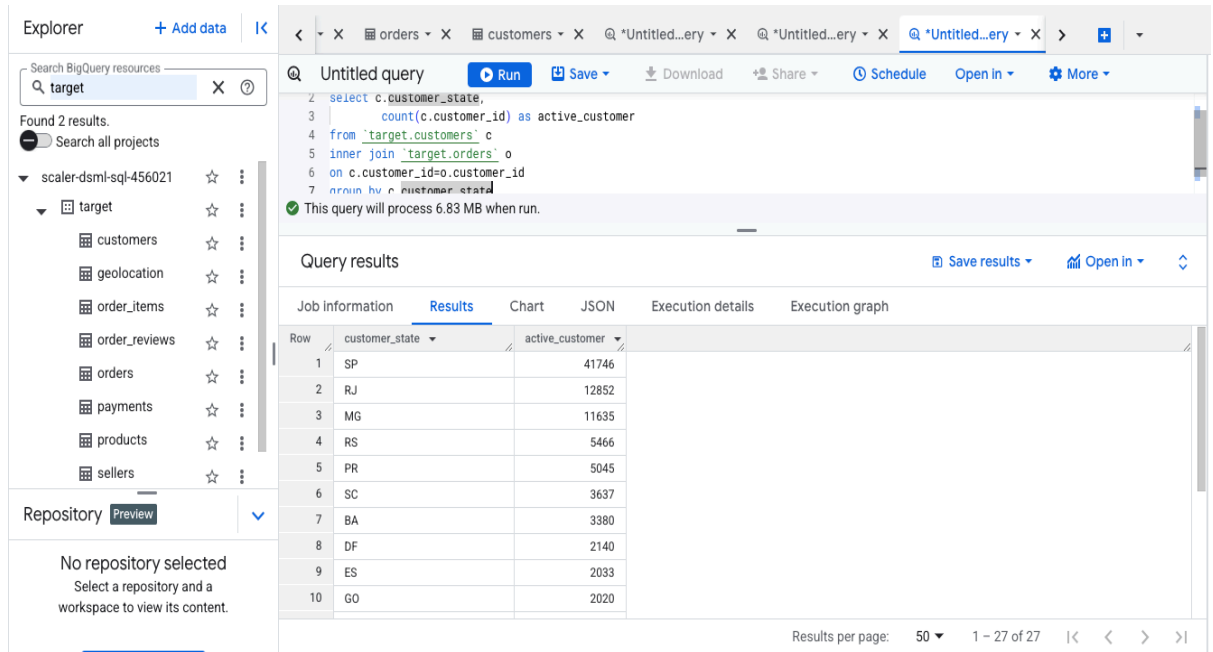
At the bottom right, it indicates 'Results per page: 50' and '1 - 27 of 27'.

(ii) Only Customers Who Have Placed at Least One Order per State

Query:

```
select c.customer_state,
       count(c.customer_id) as active_customer
from `target.customers` c
inner join `target.orders` o
on c.customer_id=o.customer_id
group by c.customer_state
order by active_customer desc
```

Screenshot of output



The screenshot displays the Google Cloud BigQuery console. On the left, the Explorer pane shows a project named 'scaler-dsml-sql-456021' with a dataset 'target' containing tables like 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'sellers'. The main editor shows a query titled 'Untitled query' that counts active customers by state. The query results are displayed in a table with 10 rows, showing states from SP to GO and their corresponding active customer counts. The interface includes tabs for Job information, Results, Chart, JSON, Execution details, and Execution graph. A message indicates the query will process 6.83 MB.

```
2 select c.customer_state,
3       count(c.customer_id) as active_customer
4 from `target.customers` c
5 inner join `target.orders` o
6 on c.customer_id=o.customer_id
7 group by c.customer_state
```

This query will process 6.83 MB when run.

Row	customer_state	active_customer
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637
7	BA	3380
8	DF	2140
9	ES	2033
10	GO	2020

Results per page: 50 | 1 - 27 of 27

Insights :

- 1) By figuring out the state wise customers and active customers it is found out that states "SP", "RJ" and "MG" has holds majority of customers
- 2) State with names "RR", "AP" and "AC" has least no. of customers.
- 3) Least customers states have the opportunities for expansion or marketing.

Recommendations:

- 1) Company can launch geo-targeted awareness or promotions for states with lower customer volume.
- 2) For High customer count states, company can strengthen operations and loyalty programs
- 3) For Low customer count states, company can launch targeted awareness campaigns, test ads, or address barriers like shipping limitation

4. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

1. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).
You can use the "payment_value" column in the payments table to get the cost of orders.

Query:

```
with cte1 as
    (select p.order_id,
            p.payment_value,
            extract(year from o.order_purchase_timestamp) as order_year,
            extract(month from o.order_purchase_timestamp) as order_month
    from `target.payments` p
    inner join `target.orders` o
    on p.order_id=o.order_id),

    cte2 as
    (select order_year,
            sum(payment_value) as cost_of_orders
    from cte1
    where order_year in (2017,2018) and order_month between 1 and 8
    group by order_year
    )

Select *,(cost_of_orders-prev_year_value)/prev_year_value*100 as
percentage_increase
from
    (select *,
            lag(cost_of_orders) over(order by order_year) as prev_year_value,

    from cte2)X
```

Screenshot of output

The screenshot displays the Google Cloud BigQuery console. On the left, the 'Explorer' pane shows a project named 'target' with various tables like 'customers', 'geolocation', 'order_items', etc. The main area shows a query titled 'Untitled query' with the following SQL code:

```

12
13 cte2 as
14 (select order_year,
15      sum(payment_value) as cost_of_orders
16   from cte1
17   where order_year in (2017,2018) and order_month between 1 and 8
18   group by order_year
19  )
20
21 Select *,(cost_of_orders-prev_year_value)/prev_year_value*100 as percentage_increase
22 from
23 (select *,
24  lag(cost of orders) over(order by order year) as prev year value.

```

Below the query, a message states: 'This query will process 8.14 MB when run.' The 'Query results' section shows a table with the following data:

Row	order_year	cost_of_orders	prev_year_value	percentage_increase
1	2017	3669022.120000...	null	null
2	2018	8694733.840000...	3669022.120000...	136.9768716466...

Insights :

- 1) 136.97 % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).
- 2) The % increase in spending by customers over the 2 years shows growth in customer base and higher average order value.

Recommendations:

- 1) For Significant increase company should consider reinforcing like pricing, product mix, customer incentives.
2. Calculate the Total & Average value of order price for each state.

Query:

```

with cte1 as
    (select p.order_id,
           p.payment_value,
           c.customer_state
     from `target.payments` p
    inner join `target.orders` o
    on p.order_id=o.order_id
    inner join `target.customers` c
    on o.customer_id=c.customer_id)

select customer_state,
       round(avg(payment_value),2) as avg_value,
       round(sum(payment_value),2) as total_value
from cte1
group by customer_state

```

Screenshot of output

The screenshot shows the Google BigQuery interface. On the left is the Explorer panel with a search for 'target' and a list of tables including customers, geolocation, order_items, order_reviews, orders, payments, products, and sellers. The main panel displays a query titled 'Untitled query' with the following SQL code:

```
inner join `target.orders` o
on p.order_id=o.order_id
inner join `target.customers` c
on o.customer_id=c.customer_id
```

Below the query, a message states: 'This query will process 14.21 MB when run.' The 'Query results' section shows a table with the following data:

Row	customer_state	avg_value	total_value
1	MG	154.71	1872257.26
2	SP	137.5	5998226.96
3	CE	199.9	279464.03
4	RS	157.18	890898.54
5	RJ	158.53	2144379.69
6	MT	195.23	187029.29
7	GO	165.76	350092.31
8	SC	165.98	623086.43
9	PR	154.15	811156.38
10	ES	154.71	325967.55

At the bottom right, it indicates 'Results per page: 50' and '1 - 27 of 27'.

Insights :

- 1) After sorting the total value it shows that State "SP" has highest order price value.
- 2) While sorting by avg. value it shows that State "PB" has the highest avg. order price value among all the states.
- 3) Total order value also denotes total money received from orders in each states.
- 4) Avg. Value per order shows the customer purchasing power.

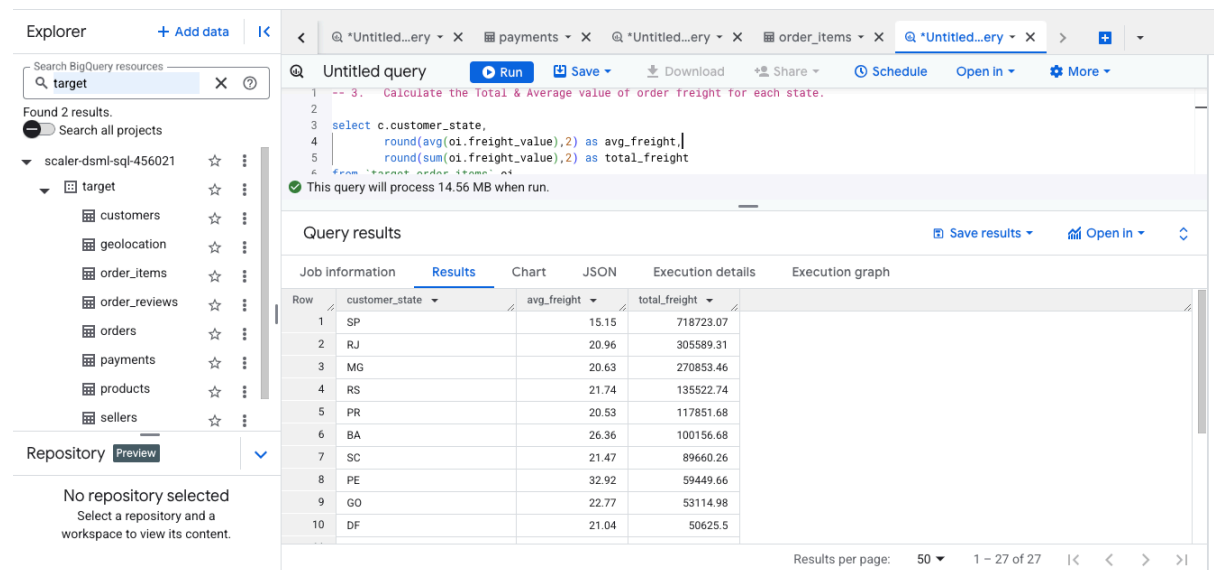
Recommendations:

- 2) For High total but low average company should streamline logistics or offer subscriptions.
3. Calculate the Total & Average value of order freight for each state.

Query:

```
select c.customer_state,
       round(avg(oi.freight_value),2) as avg_freight,
       round(sum(oi.freight_value),2) as total_freight
from `target.order_items` oi
inner join `target.orders` o
on oi.order_id=o.order_id
inner join `target.customers` c
on o.customer_id=c.customer_id
group by c.customer_state
order by total_freight desc
```

Screenshot of output



The screenshot shows the Google Cloud BigQuery interface. On the left is the Explorer pane with a search for 'target' and a list of datasets including 'target', 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'sellers'. The main pane shows a query titled 'Untitled query' with the following SQL code:

```
-- 3. Calculate the Total & Average value of order Freight for each state.
select c.customer_state,
       round(avg(o1.freight_value),2) as avg_freight,
       round(sum(o1.freight_value),2) as total_freight
from `target.order_items` as o1
join `target.customers` as c
on o1.customer_id = c.customer_id
```

Below the query is a table titled 'Query results' with the following data:

Row	customer_state	avg_freight	total_freight
1	SP	15.15	718723.07
2	RJ	20.96	305589.31
3	MG	20.63	270853.46
4	RS	21.74	135522.74
5	PR	20.53	117851.68
6	BA	26.36	100156.68
7	SC	21.47	89660.26
8	PE	32.92	59449.66
9	GO	22.77	53114.98
10	DF	21.04	50625.5

Insights :

- 1) After sorting the total value it shows that State “SP” has highest total freight value.
- 2) While sorting by avg. value it shows that State “RR” has the highest avg. freight value among all the states.

Recommendations:

- 1) For High avg. freight value company should consider offering free shipping thresholds or optimizing delivery routes.
- 2) For Low freight with Efficient delivery regions company should replicate best practices in other states as well.

5. Analysis based on sales, freight and delivery time.

1. Find the no. of days taken to deliver each order from the order’s purchase date as delivery time.
Also, calculate the difference (in days) between the estimated & actual delivery date of an order.
Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

- **time_to_deliver** = order_delivered_customer_date - order_purchase_timestamp
- **diff_estimated_delivery** = order_delivered_customer_date - order_estimated_delivery_date

Query:

```
select order_id,  
       date_diff(order_delivered_customer_date,order_purchase_timestamp, day)  
as days_to_deliver,  
  
date_diff(order_delivered_customer_date,order_estimated_delivery_date,day) as  
diff_estimated_delivery  
from `target.orders`
```

Screenshot of output

The screenshot shows the Google BigQuery interface. On the left is the Explorer panel with a search for 'target' and a list of datasets including 'target', 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'sellers'. The main panel displays a query titled 'Untitled query' with the following SQL code:

```
1 select order_id,  
2       date_diff(order_delivered_customer_date,order_purchase_timestamp, day) as days_to_deliver,  
3  
4       date_diff(order_delivered_customer_date,order_estimated_delivery_date,day) as diff_estimated_delivery  
5 from `target.orders`
```

Below the query, a status message indicates: 'This query will process 5.48 MB when run.' The 'Query results' section shows a table with 10 rows and 4 columns: 'order_id', 'days_to_deliver', and 'diff_estimated_delivery'. The 'Results' tab is selected, showing the first 10 rows of data.

Row	order_id	days_to_deliver	diff_estimated_delivery
1	65d1e226dfaeb8dc42f66542...	35	-16
2	2c45c33d2f9cb8f8b1c86cc28...	30	-28
3	1950d777989f6a877539f5379...	30	12
4	bfb0f9bdef84302105ad712d...	54	36
5	98974b076b01553d49ee6467...	43	-5
6	c4b41c36dd589e901f6879f25...	36	-14
7	d2292ff2201e74c5db154d1b7...	29	-20
8	95e01270fcbae986342340010...	30	-19
9	ed8c7b1b3eb256c70ce0c7423...	44	-5
10	5cc475c7c03290048eb2e742c...	68	18

At the bottom right, it shows 'Results per page: 50' and '1 - 50 of 99441'.

Insights :

- 1) The avg. time to deliver is 12 days and max. time to deliver is 209 days and min. time to deliver is 0 days.
- 2) Diff estimated delivery shows how early (- value) or late (+ value) the order was delivered compared to the estimated time.
- 3) Out of 96476 orders delivered, 6535 orders were delivered late.

Recommendations:

- 1) For deliveries which are often late, the company should work with logistics partners, set more realistic estimates, or improve warehouse dispatch times.

2. Find out the top 5 states with the highest & lowest average freight value.

Query: for top 5 highest avg. freight value

```
with cte1 as
    (select c.customer_state,
        round(avg(oi.freight_value),2) as avg_freight,
        from `target.order_items` oi
        inner join `target.orders` o
        on oi.order_id=o.order_id
        inner join `target.customers` c
        on o.customer_id=c.customer_id
        group by c.customer_state)

select * from
    (select customer_state,avg_freight,
        dense_rank() over(order by avg_freight desc) as
highest_freight_rank
        from cte1)X
where highest_freight_rank<=5
order by highest_freight_rank
```

Screenshot of output

The screenshot displays the BigQuery web interface. On the left, the 'Explorer' panel shows a project named 'scaler-dsml-sql-456021' with a dataset 'target' containing tables like 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'sellers'. The main editor shows a SQL query titled 'Untitled query' that uses a CTE to calculate average freight values and ranks them. Below the query, a message states 'This query will process 14.56 MB when run.' The 'Query results' section is active, showing a table with 5 rows of data. The table has columns for 'customer_state', 'avg_freight', and 'highest_freight_rank'. The results are sorted by 'highest_freight_rank' in ascending order.

Row	customer_state	avg_freight	highest_freight_rank
1	RR	42.98	1
2	PB	42.72	2
3	RO	41.07	3
4	AC	40.07	4
5	PI	39.15	5

At the bottom right, the interface indicates 'Results per page: 50' and '1 - 5 of 5'.

Query: for top 5 lowest avg. freight value

```
with cte1 as
    (select c.customer_state,
        round(avg(oi.freight_value),2) as avg_freight,
        from `target.order_items` oi
        inner join `target.orders` o
        on oi.order_id=o.order_id
        inner join `target.customers` c
        on o.customer_id=c.customer_id
        group by c.customer_state)

select * from
    (select customer_state,avg_freight,
        dense_rank() over(order by avg_freight ) as
lowest_freight_rank
        from cte1)X
where lowest_freight_rank<=5
order by lowest_freight_rank
```

Screenshot of output

The screenshot displays a BigQuery workspace. On the left, the 'Explorer' panel shows a project named 'scaler-dsm-sql-456021' with a table named 'target'. The main area shows a query editor with the following SQL code:

```
with cte1 as
    (select c.customer_state,
        round(avg(oi.freight_value),2) as avg_freight,
        from `target.order_items` oi
        inner join `target.orders` o
        on oi.order_id=o.order_id
        inner join `target.customers` c
        on o.customer_id=c.customer_id
        group by c.customer_state)

select * from
    (select customer_state,avg_freight,
        dense_rank() over(order by avg_freight ) as
lowest_freight_rank
        from cte1)X
where lowest_freight_rank<=5
order by lowest_freight_rank
```

Below the query editor, a message states: 'This query will process 14.56 MB when run.' The 'Query results' section shows a table with the following data:

Row	customer_state	avg_freight	lowest_freight_rank
1	SP	15.15	1
2	PR	20.53	2
3	MG	20.63	3
4	RJ	20.96	4
5	DF	21.04	5

The interface also includes a 'Repository' section on the left with a message: 'No repository selected. Select a repository and a workspace to view its content.'

Insights :

- 1) The Highest 5 States where customers pay the most for shipping.
- 2) The Lowest 5 States with lowest shipping costs on average

Recommendations:

- 1) Company should focus on minimizing the gap between the highest and lowest avg. freight value.

- Find out the top 5 states with the highest & lowest average delivery time.

Query: for top 5 highest avg. delivery time

```
with cte1 as
(
select
    c.customer_state,

round(avg(date_diff(o.order_delivered_customer_date,o.order_purchase_timestamp, day)),2) as days_to_deliver
    from `target.orders` o
    inner join `target.customers` c
    on o.customer_id=c.customer_id
    group by c.customer_state
    order by days_to_deliver desc)

select * from
    (select *,
        dense_rank() over(order by days_to_deliver desc) as
delivery_rank
        from cte1)X
where delivery_rank<=5
order by delivery_rank
```

Screenshot of output

The screenshot displays the Google Cloud BigQuery interface. On the left, the 'Explorer' pane shows a project named 'scaler-dsml-sql-456021' with a dataset 'target' containing tables like 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'sellers'. The main editor shows an 'Untitled query' with the SQL code from the previous block. Below the editor, a message states 'This query will process 8.32 MB when run.' The 'Query results' section is active, showing a table with 5 rows and 4 columns: 'customer_state', 'days_to_deliver', 'delivery_rank', and an unnamed column. The results are as follows:

Row	customer_state	days_to_deliver	delivery_rank
1	RR	28.98	1
2	AP	26.73	2
3	AM	25.99	3
4	AL	24.04	4
5	PA	23.32	5

At the bottom right, it indicates 'Results per page: 50' and '1 - 5 of 5'.

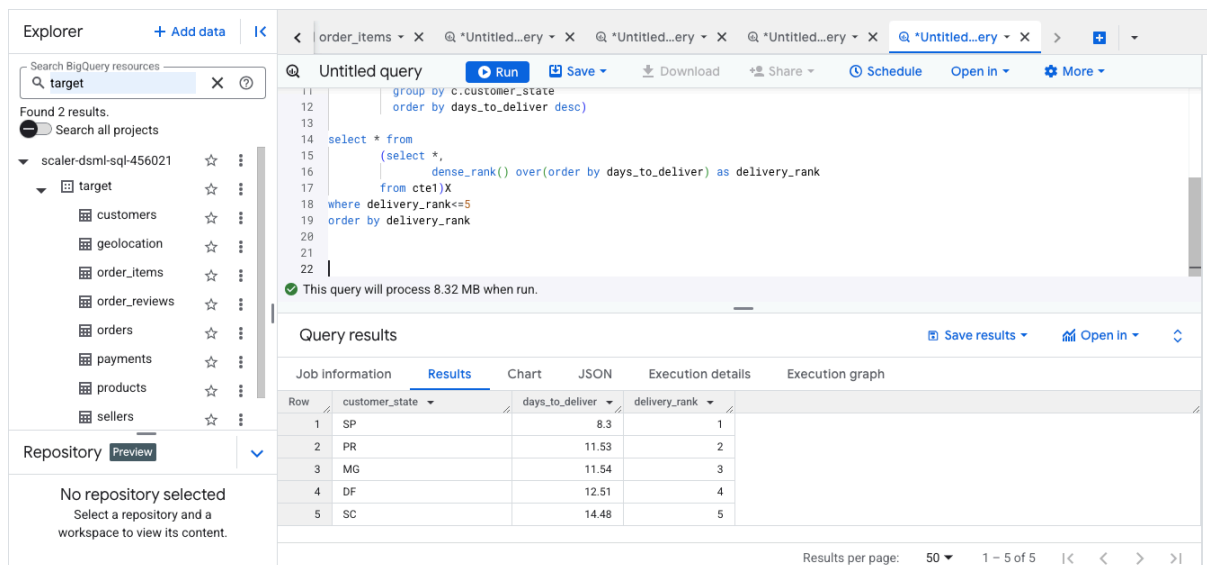
Query: for top 5 lowest avg. delivery time

```
with cte1 as
(
    select
        c.customer_state,

        round(avg(date_diff(o.order_delivered_customer_date,o.order_purchase_timestamp,
        day)),2) as days_to_deliver
        from `target.orders` o
        inner join `target.customers` c
        on o.customer_id=c.customer_id
        group by c.customer_state
        order by days_to_deliver desc)

select * from
    (select *,
        dense_rank() over(order by days_to_deliver) as delivery_rank
        from cte1)X
where delivery_rank<=5
order by delivery_rank
```

Screenshot of output



The screenshot shows a SQL query editor interface. On the left is an Explorer pane with a tree view of a database schema. The main area displays a SQL query and its results. The query is the same one as provided in the text. Below the query, a message states 'This query will process 8.32 MB when run.' The 'Query results' section shows a table with 5 rows and 4 columns: Row, customer_state, days_to_deliver, and delivery_rank. The results are as follows:

Row	customer_state	days_to_deliver	delivery_rank
1	SP	8.3	1
2	PR	11.53	2
3	MG	11.54	3
4	DF	12.51	4
5	SC	14.48	5

Insights :

- 1) The States named "RR" takes the highest delivery time
- 2) The States named "SP" takes the lowest delivery time

Recommendations:

- 1) Company should focus on minimizing the gap between the highest and lowest delivery days between the states since the gap of days is high.

4. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.
You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

Query:

```
with cte1 as
(
select
    c.customer_state,

round(avg(date_diff(o.order_delivered_customer_date,o.order_estimated_delivery_date
,day)),2) as
    avg_diff_estimated_delivery
from `target.orders` o
inner join `target.customers` c
on o.customer_id=c.customer_id
group by c.customer_state
order by avg_diff_estimated_delivery desc)

select * from
    (select *,
        dense_rank() over(order by avg_diff_estimated_delivery ) as
fast_delivery_rank
    from cte1)X
where fast_delivery_rank<=5
order by fast_delivery_rank
```

Screenshot of output

The screenshot displays the Google BigQuery interface. On the left, the 'Explorer' pane shows a project named 'scaler-dsml-sql-456021' with a dataset named 'target'. The 'target' dataset contains tables: customers, geolocation, order_items, order_reviews, orders, payments, products, and sellers. The main editor shows a SQL query titled 'Untitled query' that uses a CTE to calculate the average difference between actual and estimated delivery dates for each state, then ranks the states by this difference. A message indicates the query will process 8.32 MB. Below the query, the 'Query results' section shows a table with 5 rows and 4 columns: Row, customer_state, avg_diff_estimated_delivery, and fast_delivery_rank. The results are as follows:

Row	customer_state	avg_diff_estimated_delivery	fast_delivery_rank
1	AC	-19.76	1
2	RO	-19.13	2
3	AP	-18.73	3
4	AM	-18.61	4
5	RR	-16.41	5

At the bottom right, it shows 'Results per page: 50' and '1 - 5 of 5'.

Insights :

- 1) The States named "AC" has the fastest avg. delivery time
- 2) The avg. diff estimated date will be negative that means how many days early on average the orders are delivered.
- 3) More negative value the better the state is performing vs estimated delivery time.

Recommendations:

- 1) Company should focus on logistics operations in those states with higher delivery time to improve delivery performance.
- 2) Company can consider offering next-day delivery as a premium in these high-performing states.

6. Analysis based on the payments:

1. Find the month on month no. of orders placed using different payment types.

Query:

```
with cte1 as
(
select p.order_id,
       p.payment_type,
       extract(month from o.order_purchase_timestamp) as order_month
from `target.payments` p
inner join `target.orders` o
on p.order_id=o.order_id)

select order_month,payment_type,count(order_id)as no_of_orders from cte1
group by order_month,payment_type
order by order_month,payment_type
```

Screenshot of output

The screenshot shows the Google BigQuery interface. On the left is the Explorer pane with a search for 'target' and a list of datasets including 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'sellers'. The main area displays an 'Untitled query' with the SQL code from the previous block. Below the query, it states 'This query will process 8.47 MB when run.' The 'Query results' section is active, showing a table with 10 rows and 3 columns: 'order_month', 'payment_type', and 'no_of_orders'. The results are as follows:

Row	order_month	payment_type	no_of_orders
1	1	UPI	1715
2	1	credit_card	6103
3	1	debit_card	118
4	1	voucher	477
5	2	UPI	1723
6	2	credit_card	6609
7	2	debit_card	82
8	2	voucher	424
9	3	UPI	1942
10	3	credit_card	7707

At the bottom right, it indicates 'Results per page: 50' and '1 - 50 of 50'.

Insights :

- 1) Highest no. of orders were placed by the payment type= "Credit Card"
- 2) 3 orders were placed with No Payment type that is "not defined"
- 3) Month August sees highest no. of orders placed with no. of order counts as 11248
- 4) While September Month sees lowest no. of orders placed with no. of order counts as 4535

Recommendations:

- 1) Company can tailor payment method promotions around high-usage periods (e.g., vouchers in holidays).
- 2) Company should focus on Months with low no. of orders places like Sept, Oct, Nov and Dec by giving some discounts over different payments type.

2. Find the no. of orders placed on the basis of the payment installments that have been paid.

Query:

```
select payment_installments,  
       count(distinct order_id) as no_of_orders  
from `target.payments`  
group by payment_installments
```

Screenshot of output

The screenshot displays the Google BigQuery web interface. On the left, the 'Explorer' panel shows a search for 'target' with two results: 'customers' and 'payments'. The 'payments' table is selected. The main editor shows a SQL query: `select payment_installments, count(distinct order_id) as no_of_orders from `target.payments` group by payment_installments`. Below the query, a message states: 'This query will process 4.16 MB when run.' The 'Query results' section shows a table with 10 rows and 2 columns: 'payment_installment' and 'no_of_orders'. The results are as follows:

Row	payment_installment	no_of_orders
1	0	2
2	1	49060
3	2	12389
4	3	10443
5	4	7088
6	5	5234
7	6	3916
8	7	1623
9	8	4253
10	9	644

At the bottom right, the status bar indicates 'Results per page: 50' and '1 - 24 of 24'.

Insights :

- 1) payment installments = 1 shows Orders paid in full upfront.
- 2) Orders which are split into multiple payments are mostly credit card type

Recommendations:

- 1) Company can promote use of 1-installment payments and suggests customers to pay in full by offering a discount to encourage this further.
- 2) Moreover credit and debit card one-time payment offers can be provided.